



Prueba de desempeño del módulo JavaScript

Esta prueba está diseñada para evaluar tus habilidades en desarrollo web mediante la creación de una aplicación completa que incorpore varias funcionalidades clave.

Objetivo de la prueba

Desarrollar una **Single Page Application (SPA)** utilizando JavaScript (Vanilla JS o Vite), implementando autenticación, manejo de roles, persistencia de sesión y operaciones CRUD consumiendo una API simulada con json-server o externa.

La prueba busca evaluar conocimientos técnicos en:

- JavaScript básico y avanzado
- Manipulación del DOM
- Enrutamiento
- Consumo de APIs
- Persistencia de sesión
- Manejo de roles y autorización
- Arquitectura de aplicaciones SPA
- Buenas prácticas de desarrollo
- Documentación técnica
- Navegación dinámica sin recarga de página

Contexto del problema

Una cadena de cines desea modernizar su sistema de gestión de funciones y reservas de entradas.

Actualmente, la administración de las funciones se realiza de manera manual, lo que ha generado problemas como:



- Sobreventa de asientos.
- Dificultad para consultar la disponibilidad de funciones.
- Falta de control sobre las reservas realizadas por los clientes.
- Poca visibilidad para los administradores sobre la ocupación de las salas.

Para solucionar esta situación, se requiere desarrollar una plataforma web que permita a los usuarios reservar entradas para diferentes funciones de cine y a los administradores gestionar la cartelera, las funciones y las reservas.

Roles del sistema

Administrador (admin)

Puede:

- Ver todas las reservas.
- Crear funciones de cine.
- Editar funciones.
- Eliminar funciones.
- Aprobar o cancelar reservas.
- Gestionar salas de cine (opcional con puntos extra).
- Ver todos los usuarios registrados (opcional con puntos extra).
- Consultar estadísticas de ocupación (opcional con puntos extra).

Usuario (user)

Puede:

- Consultar la cartelera disponible.
- Reservar entradas para una función.
- Ver únicamente sus reservas.
- Cancelar sus reservas.
- Modificar reservas siempre que la función no haya comenzado.

Restricciones:



- No puede gestionar funciones.
- No puede gestionar salas.
- No puede visualizar reservas de otros usuarios.
- No puede acceder a módulos administrativos.

Requerimientos funcionales

1. Autenticación

La aplicación debe incluir:

- Formulario de inicio de sesión.
- Validación de credenciales contra json-server.
- Manejo de errores de autenticación.
- Persistencia de sesión mediante:
 - localStorage
 - sessionStorage

Debe existir al menos:

- 1 usuario administrador.
- 2 usuarios estándar.

Ejemplo de estructura:

2. Gestión de Funciones

Cada función debe contener:

- ID
- Película
- Sala
- Fecha



- Hora
- Capacidad total
- Cupos disponibles
- Estado (Activa / Cancelada)

3. Gestión de Reservas

- Cada reserva debe contener:
- ID
- Usuario
- Función seleccionada
- Cantidad de entradas
- Fecha de reserva
- Estado
- Estados posibles:
 - Pendiente
 - Confirmada
 - Cancelada

Reglas de negocio

- No se pueden reservar más entradas de las disponibles.
- Una función cancelada no puede recibir nuevas reservas.
- Un usuario solo podrá modificar reservas activas.
- Las reservas canceladas no pueden reactivarse.
- El administrador puede modificar cualquier reserva.
- El sistema debe actualizar automáticamente los cupos disponibles cuando se cree o cancele una reserva.

4. CRUD de Reservas

Implementar:



- GET
- POST
- PUT/PATCH
- DELETE

Permisos

Admin

- CRUD completo sobre cualquier reserva.

User

- Crear reservas.
- Ver únicamente sus reservas.
- Editar reservas propias.
- Cancelar reservas propias.

5. Gestión de Salas (Opcional con puntos extra)

- Cada sala debe contener:
 - ID
 - Nombre
 - Capacidad
 - Tipo (2D, 3D, IMAX)
 - Estado
- Solo el administrador podrá:
 - o Crear salas.
 - o Editar salas.
 - o Eliminar salas.
 - o Consultar todas las salas.

Si decides implementar este módulo, las funciones deben estar asociadas a una sala existente.

6. SPA (Single Page Application)

La aplicación debe:



- Navegar sin recargar la página.
- Utilizar renderizado dinámico mediante JavaScript.
- Implementar enrutamiento basado en hash o History API.

7. Persistencia de sesión

Debe implementarse mediante:

- localStorage o sessionStorage.

Requisitos:

- Mantener sesión al refrescar la página.
- Logout funcional.
- Limpieza completa de datos de sesión.

8. Interfaz de usuario

Puede utilizarse:

- CSS
- Bootstrap
- TailwindCSS
- Material UI
- Cualquier librería de estilos

Se evaluará:

- Organización visual.
- Experiencia de usuario.
- Diseño responsive (no obligatorio).
- Claridad de navegación.

9. Requisitos técnicos

Debe incluir:

- Arquitectura modular.
- Manipulación del DOM.



- Manejo de eventos.
- Consumo de APIs mediante Fetch o Axios.
- Manejo de errores.
- Código limpio y reutilizable.
- Protección de rutas mediante middleware o guards simulados.

10. Documentación obligatoria (README.md en inglés)

Debe incluir:

- Project name
- Description
- Technologies used
- Installation
- Running the project
- Running json-server
- Test users
- Project structure
- Role permissions
- Technical decisions

Bonus (Opcional)

- Dashboard con estadísticas.
- Dark mode.
- Buscador.
- Filtros por fecha.
- Paginación.
- Notificaciones Toast.
- Deploy en Vercel, Netlify o cualquiera de tu preferencia.



Criterio	Porcentaje
Cumplimiento de requerimientos	15%
Comprensión de JavaScript	10%
Enrutamiento SPA	15%
Persistencia y manejo de sesión	10%
Documentación	10%
Sustentación técnica	40%

Nota importante

Para el desarrollo de la prueba se les entregará una base inicial del proyecto que incluirá:

- Proyecto configurado con Vite
- TailwindCSS instalado y funcionando
- Configuración base de json-server
- Estructura mínima de carpetas
- Navegación básica/redireccionamiento inicial

Nota: puede presentar algunos errores al levantar que deberás corregir y sustentar por qué fallaba y cómo se corrige.

La base entregada NO incluirá:

- autenticación
- manejo de sesión
- control de acceso
- protección de rutas
- lógica de roles
- guards
- persistencia de usuario



Un mensaje para ustedes

Esta prueba no marca el final de la aventura en Riwi. Por el contrario, representa un punto de partida para lo que viene en las siguientes etapas de formación y para los retos que encontrarán en escenarios reales de desarrollo.

El objetivo no es únicamente entregar una solución funcional, sino demostrar su capacidad para analizar problemas, tomar decisiones técnicas y construir software de manera organizada. A partir de este momento comienza un proceso de perfilamiento donde identificaremos fortalezas, oportunidades de mejora y áreas en las que cada uno puede seguir creciendo.

La base entregada busca optimizar el tiempo disponible y permitirles enfocarse en la lógica, la arquitectura y la resolución de problemas. Sin embargo, son libres de adaptar, reorganizar o incluso reestructurar completamente el proyecto si consideran que existe una mejor forma de resolverlo.

No se evaluará quién siga exactamente la misma estructura entregada, sino quién logre cumplir de manera efectiva los requerimientos planteados, justificando sus decisiones técnicas y demostrando comprensión de lo que está construyendo.

Recuerden que en el desarrollo de software no siempre existe una única solución correcta. Lo importante es que la solución propuesta sea coherente, mantenible y responda a las necesidades del problema.

Confíen en lo que han aprendido hasta ahora, gestionen bien su tiempo y no tengan miedo de tomar decisiones. Esta prueba es una oportunidad para demostrar todo lo que han construido durante su proceso de formación.

¡Good Luck!



www.riwi.io



301 732 53 27



Cl. 16 # 55 - 129

